

# Ismerkedés a mikrovezérlőkkel

Mérést végezte: Gillich Ádám

Mérés dátuma: 2026. szeptember 29.

Jegyzőkönyv végleges változata: 2026. szeptember 30.

## 1. A mérés célja

A mérés célja az volt, hogy néhány egyszerű program segítségével bepillantást nyerjünk a mikrokontrollerek világába és az azokban megszerzett tudást egy szabadon választott programban alkalmazni, ami akár több példa anyagát is felhasználva. Ezenkívül megismerni az oszcilloszkópot is, és annak az áramkörbe kötését.

## 2. A kötelező feladatok oszcillogramjai

### 2.1. Digitális I/O műveletek

Elvégeztem a feladatot a leírás szerint és a LED villogott.

### 2.2. PWM – Pulse-Width Modulation

A feladatot a leírás szerint elvégeztem, és a LED lassan villogott. Halványan kezdve, és lassan felerősödve, majd visszahalványodva villogott.

Az oszcilloszkópon a négyzögjel látványosan jelent meg és növekedett, majd csökkent, viszont mivel az oszcilloszkópot nem lehet folyamatos felvételkészítésre bírni, hanem csak egy-egy állapotról tud képet készíteni, ezért az `alpha` változó manuális beállításával készítettem több képet több különböző állapotról.

A 2 ábrán látszik, hogy `alpha` értéke 0 – 255 közt változhat, ami a négyzögjel "kitöltöttségét" szabályozza, 0 – 100%-ig, `alpha` a kitöltéssel arányos és a köztes értékek kitöltöttsége követi a  $\alpha/255$  arányt. A kódban minden ciklusban ez egyenként volt változtatva így elérve a LED fényerejének változását. Illetve az is látszik, hogy "szőrös" a jel LOW, 0 V, környékén, illetve ugyanez HIGH,  $\approx 5$  V, esetében, habár nem ad le pontosan 5 V-ot, hanem valamennyivel kevesebbet. Ez több ok miatt is lehet, például azért, mert a nulla az oszcilloszkópon nem pontosan lett beállítva, mint az a legfelső képről is látszik, valamint a rendszer terhelés alatt van, ami egy kicsit csökkentheti a leadott feszültséget.

Igazából ebből szépen kijön, miért kell hirtelen nagy ugrásokkal kezelni az ilyen eszközöket, hogy meg legyen a bináris funkció ("van-e áram vagy nincs"), hiszen ha a legkisebb változásra is reagálnának az eszközök, akkor nem lehetne megbízhatóan kezelni őket.

### 2.3. ADC és soros vonal írása

Összeállítottam a kísérletet a leírás szerint, majd az IDE Monitorát akartam használni, viszont az folyamatosan null értékeket írt ki, ami azért volt, mert a leírás által adott értéket a `monitor_speed`-re át kellett írni 115200-ról 9600-ra, amire a `Serial.begin(9600)` be volt állítva a forráskódban.

Ezután a potméter adatküldése működött.

### 2.4. Folyadékkristályos kijelző illesztése

Az LCD kijelző DAT, adat és CLK, órajel kimenetét nem az A4 és A5 kimenetekbe kellett bekötni, hanem az Arduino SDA és SCL kivezetésébe. A breadboard kocogtatásával lehetett új kockát dobni. (érzékeny volt a tilt-switch)

Az oszcilloszkópon, ha teljesen kizoomoltunk volna, akkor egy hatalmas blokknyi jelet láttunk volna, mivel ezen nehéz bármit is észrevenni, inkább jól belenagyítottam, így kapva a 3 és 4 ábrákat. Amiken alul narancssárgán van az órajel, és felül kéken az adat. Az órajel folyamatosan küld egy egységes jelet mindig ugyanazzal a periódussal, és ugyanazzal az alakkkal, aminek a felfutó oldalát a felhúzó ellenállás "simítja", tehát szépen felfut, nem pedig hirtelen felugrik, mint a PWM esetében a 2 ábrán. Minden ilyen felfutó élt bejelöltem függőleges piros vonalakkal, mert az órajel minden felfutásánál lesz az adatjel olvasva, ezeknek az értékét is megjelöltem, egyesekkel és nullákkal.

Azért van szükség a két jelre, mert az adat nem tartalmazza azt, hogy mikor kezdődött a jel, és mikor ér véget, valamint azt sem tudná, hol ért véget egy bit. Ezért a 4 ábrán az első 4 nulla lehetne bármennyi, illetve lehetne az első egyes az egész adatsor legeleje. De mivel ott az órajel is, ezért az megmondja, mikor kell leolvasni az adatokat, biteket csinálva belőlük.

Még azt is megfigyelhetjük, hogy mindig az órajellel szinte együtt, egy kicsit késleltetve vág le az adat, és hogy akkor kezd el felfutni, mikor az előző órajel levágott, tehát az SDA akkor változhat, amikor az SCL LOW. Ezenkívül, ha olvasni kell, akkor a két jel együtt marad konstans (az adat HIGH vagy LOW attól függően mi a bit). Az adatjel akkor kezd el

változni, amikor az órajel LOW, hogy mikor tényleg be kell olvasni az adatot, biztosan HIGH-ra kerüljön a feszültség. Az órajel periódusideje  $T \approx 10 \mu s$ ,  $f \approx 100 \text{ kHz}$  szemmel leolvasva.

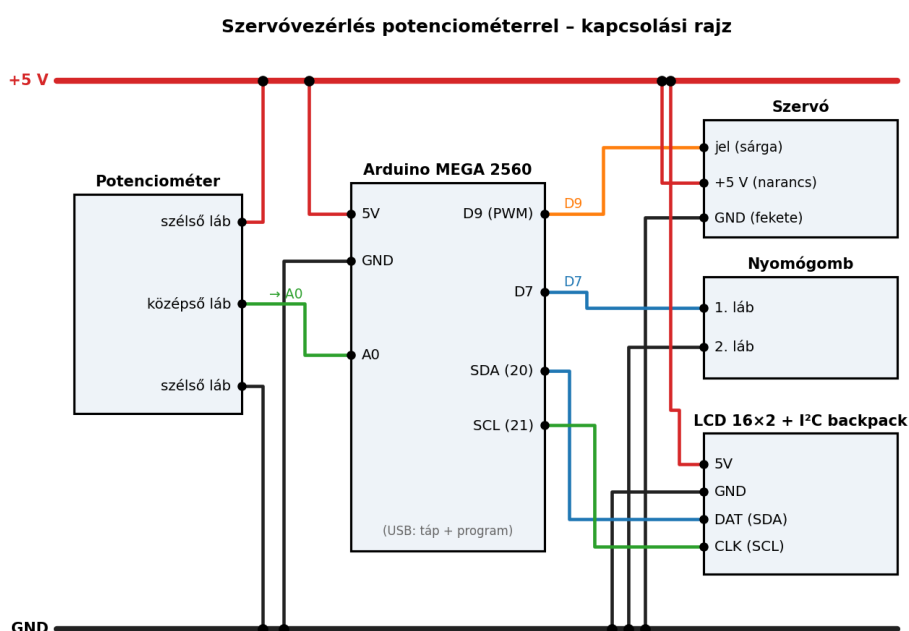
### 3. Választott feladat dokumentációja

A választott feladatom a Szervó vezérlés volt.

A feladat az összes példát felhasználta, és adott hozzá egy szervó motort. A szervó motort a potméter állítja, aminek az irányát gombnyomásra meg lehet fordítani. Ezenkívül az LCD kijelzőn a szögelfordulás és a forgás iránya lett kiírva.

#### 3.1. Az összeállításhoz szükséges eszközök, és kapcsolási ábrájuk

- Potenciométer
- Nyomógomb
- LCD kijelző
- Szervó motor



1. ábra. Az összeállítás kapcsolási ábrája.

#### 3.2. A program működésének leírása

Itt most csak a program fő `loop()` függvényét magyarázom el, de az 1. lista, tartalmazza az egész forráskódot, kommentekkel ellátva, hogyha más részletekre is kíváncsi az olvasó. Illetve az mi hova lett bekötve megtalálható az 1. ábrán.

A program ciklusa először beolvassa a gombot, hogy változott-e az állapota, majd ha igen (egy kis ráhagyással,  $50 \text{ ms}$ ), akkor két dolog történhet, ha HIGH-ra változott, azt jelenti, hogy a gomb fel lett engedve, nincs lenyomva, így csak átállítjuk a gomb állapotát (`gombElozo` változóval) ha pedig LOW-ra változott, akkor az azt jelenti, hogy lenyomtuk, így meg kell fordítani az irányt is.

Ezután a `map()` függvénnyel átskálázzuk a potméter  $0 - 1023$  skáláját  $0 - 180$ -ra, hogy szöveget tudjunk megadni a szervó motornak.

Majd csak akkor, ha változás történt a szögben, odaforgatjuk a szervó motort és kiírjuk a változást az LCD-re.

És egy kis késleltetéssel zárul a kör.

Amit még megjegyeznék, az az, hogy a zaj csökkentésének érdekében a potméter állását 8 minta átlagából számítja ki a program.

Lista 1. A szervóvezérlő program forráskódja

```

1  #include <Arduino.h>
2
3  #include "Wire.h"
4  #include "Adafruit_LiquidCrystal.h"
5  #include <Servo.h>
6
7  const int pinPOTM = A0;           // potenciometer kozepso laba
8  const int pinSZERVO = 9;         // szervo jelvezeteke
9  const int pinGOMB = 7;           // iranyvalto nyomogomb
10
11  const int ALAP = 90;              // alapallapot: kozepallas fokban
12  const unsigned long PERGES = 50; // gomb pergesmentesitesi ideje ms
13
14  Adafruit_LiquidCrystal lcd(0);    // i2c, 0-s cim
15  Servo szervo;
16
17  bool forditott = false;           // megforditott iranyu mukodes
18  int gombElozo = HIGH;             // gomb elozo (elfogadott) allapota
19  unsigned long gombIdo = 0;        // utolso elfogadott gombvaltas ideje
20  int szogElozo = -1;              // utoljara beallitott szog
21
22
23  int olvasPot() {                  // 8 minta atlaga a zaj ellen
24      long osszeg = 0;
25      for (int i = 0; i < 8; i++) {
26          osszeg += analogRead(pinPOTM);
27      }
28      return osszeg / 8;
29  }
30
31  void kiir(int szog) {              // szog es irany kiirasa az LCD-re
32      int elteres = szog - ALAP;     // alapallapothoz viszonyitott szog
33      lcd.setCursor(0, 0);
34      lcd.print("Szog: ");
35      if (elteres > 0) {
36          lcd.print('+');
37      }
38      lcd.print(elteres);
39      lcd.print((char)223);           // fok jel az LCD karakterkészletében
40      lcd.print(" ");                // elozo, hosszabb szam maradékanak torlese
41      lcd.setCursor(0, 1);
42      lcd.print(forditott ? "Irany: forditott" : "Irany: normal  ");
43  }
44
45  void setup() {
46      pinMode(pinGOMB, INPUT_PULLUP); // lenyomva LOW, elengedve HIGH
47      szervo.attach(pinSZERVO);
48      lcd.begin(16, 2);               // LCD 16 karakter mindket sorban
49      lcd.setBacklight(HIGH);
50  }
51
52  void loop() {
53      int gomb = digitalRead(pinGOMB);
54      if (gomb != gombElozo && millis() - gombIdo > PERGES) {
55          gombIdo = millis();
56          gombElozo = gomb;
57          if (gomb == LOW) {           // lenyomas pillanataban fordulunk
58              forditott = !forditott;
59              szogElozo = -1;         // kiiras frissitesenek kikenyszeritese
60          }
61      }
62
63      int szog = map(olvasPot(), 0, 1023, 0, 180); // 0..1023 -> 0..180 fok
64      if (forditott) {
65          szog = 180 - szog;
66      }
67
68      if (szog != szogElozo) {        // csak valtozaskor irunk, nem villog
69          szervo.write(szog);
70          kiir(szog);
71          szogElozo = szog;
72      }
73      delay(15);                      // a szervonak ido kell a mozgashoz
74  }

```

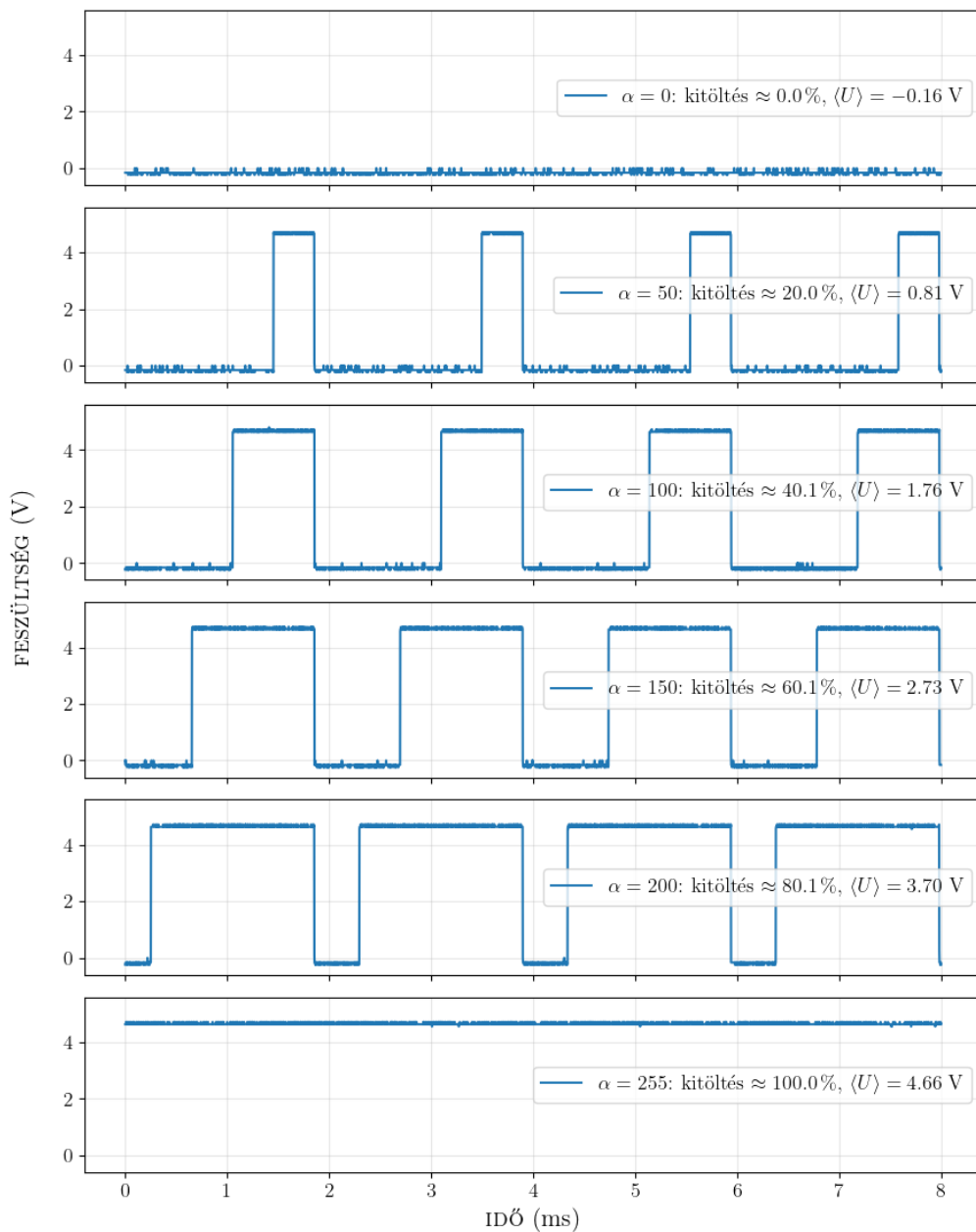
## Összegzés

Összességében ez egy rövid megismerkedés volt a mikrokontrollerek világával. Ami bemutatott néhány egyszerűbb, de sokat használt eszközt és annak a kezelését, majd egy választható problémán keresztül a megszerzett tudást alkalmazni is lehetett.

És a mikrokontrollereken kívül az oszcilloszkóp használatával is megismerkedik az ember.

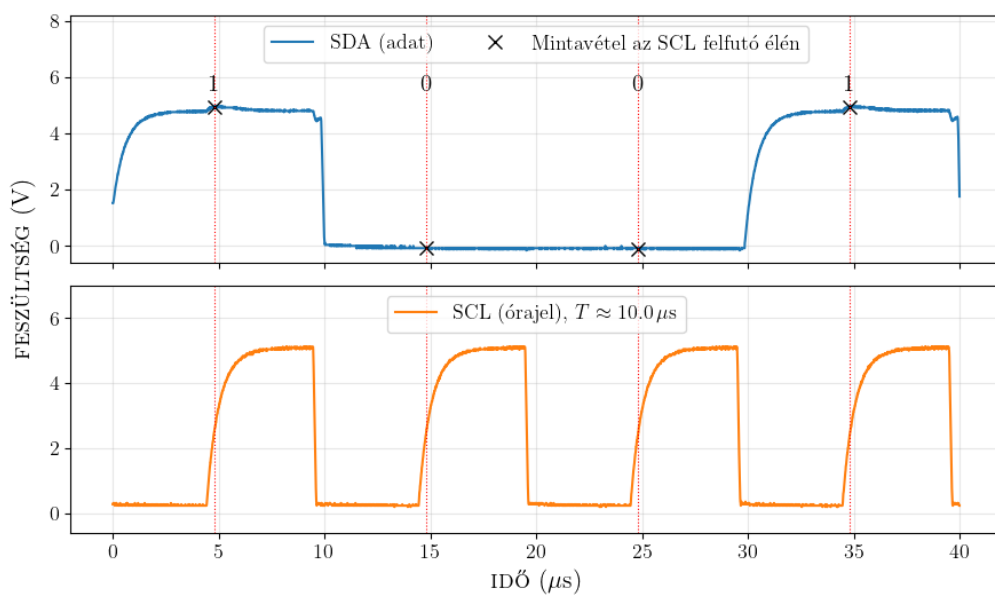
## Ábrák

### A LED meghajtójele (D11, PWM)



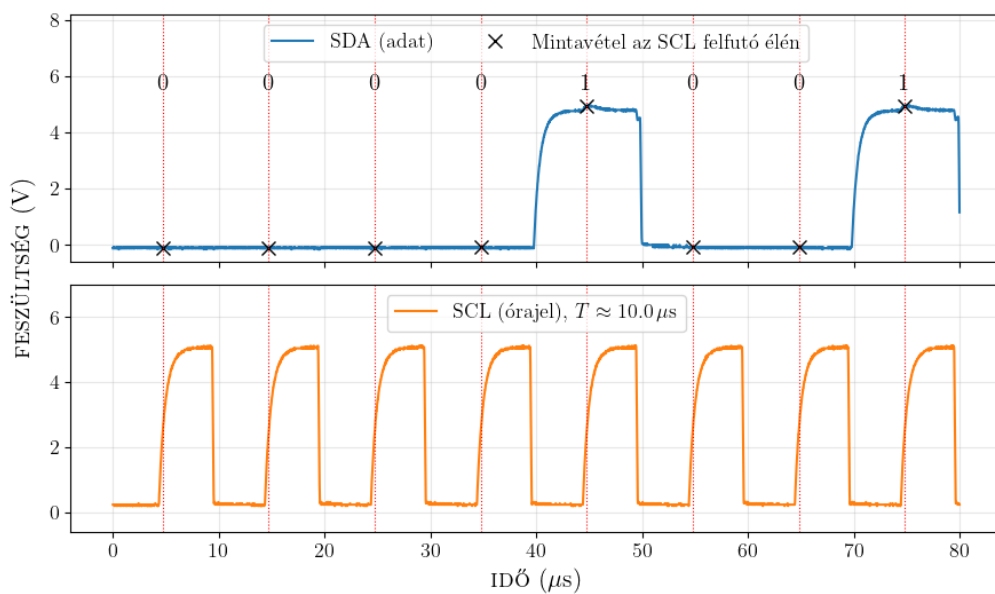
2. ábra. A LED meghajtójele különböző alpha értékeknél. Valamint  $\langle U \rangle$  átlagfeszültséggel.

### Az I<sup>2</sup>C busz jelalakja (LCD)



3. ábra. Az I<sup>2</sup>C busz jelalakja (SDA és SCL), 40  $\mu s$ -os ablak.

### Az I<sup>2</sup>C busz jelalakja (LCD)



4. ábra. Az I<sup>2</sup>C busz jelalakja (SDA és SCL), 80  $\mu s$ -os ablak.